

PULSE

Architecture Review & Recommendations

Technical review summary — March 24, 2026

Covers: Tech Stack, Frontend, Backend, Database, Pipeline, Chat, Personalization, Taxonomy, and RAG Assessment

What's Working Well

Tech Stack

The stack is pragmatic and avoids over-engineering. Key strengths:

- MapLibre GL over Mapbox saves licensing costs; CartoDB Positron tiles are clean for an event map
- Supabase (PostgreSQL + PostGIS) is the right DB — relational with geospatial, no unnecessary vector complexity
- Next.js on Vercel with serverless API routes is appropriate for beta scale
- Claude Haiku for high-volume categorization, Sonnet for vision — smart model selection
- Correctly avoided: vector DB, Redis, Kubernetes, embeddings, message queues

Scraper Pipeline

- 21 sources covering Berlin's event landscape comprehensively
- kulturdaten.berlin as backbone (~14K events) supplemented by niche venue-specific scrapers
- SHA256 fingerprint dedup prevents cross-source duplicates
- Geocoding cascade achieves 98.9% coordinate coverage

Taxonomy — 3-Layer Categorization

- Layer 1 (source tags) + Layer 2 (3,000+ keyword mappings) handle 70–80% of events at zero LLM cost
- Layer 3 (Claude Haiku) catches the rest in batches of 10 with prompt caching
- Self-learning keyword expansion: Claude discovers German terms, saves them, future events skip the LLM
- 9 categories are well-chosen for Berlin specifically

Database Design

- events_with_coords view with COALESCE fallback is a clean pattern
- Right indexes: PostGIS GiST, composite time, category+active, fingerprint uniqueness
- Row-Level Security properly separates public reads from pipeline writes

Recommendations

1. Frontend & UX

Make the Map Public Without Login

HIGH PRIORITY: Biggest UX win. Let users feel the value before asking them to sign up.

The app redirects unauthenticated users to /login. Make map and browsing fully public. Gate only bookmarks, chat, and flyer scanning behind auth.

Add PWA / Offline Capability

For a “walking around Berlin” use case, service worker caching of recently viewed events and map tiles matters. Users lose connectivity in the U-Bahn.

Viewport-Based Event Loading

As the DB grows, fetch only events within current map bounds via PostGIS ST_Within queries with a buffer. Don’t load all weekly events at once.

2. Backend & API

Add Caching on Event Endpoints

HIGH PRIORITY: Easiest performance win. Events don’t change more than once a day.

Set Cache-Control headers with stale-while-revalidate on /api/events (5–10 min TTL). Cuts Supabase read load dramatically.

Rate Limit AI Endpoints

/api/chat and /api/events/scan both call the Anthropic API. Add per-IP rate limiting to prevent cost spikes.

Watch Vercel Timeout Limits

Chat SSE streaming can hit Vercel’s 25s (Hobby) or 60s (Pro) timeout. Monitor and consider Edge Functions if needed.

3. Database

Event Expiration / Archival

HIGH PRIORITY: ~3,000 events/week = 150K+ in a year. Soft-delete or archive old events.

Keep the active query set lean. Past events should move to an archive table or get a status flag change.

Consolidate the Geocoding Dict

The hardcoded Python dict of 160+ venues is two sources of truth alongside the venues table. Make the venues table the single lookup, Nominatim the only external fallback.

Add pg_trgm for Fuzzy Search

Current ILIKE search is basic. pg_trgm gives typo-tolerant matching without a separate service. Longer term, add Meilisearch for instant fuzzy search.

4. Scraper Pipeline

Parallelize Scraper Execution

HIGH PRIORITY: 21 scrapers running sequentially in one 45-min job is fragile.

One hung scraper blocks everything. Options: separate GitHub Actions jobs per tier, move to Inngest (handles retries, timeouts, concurrency natively), or a lightweight worker on Railway/Fly.io.

Add Monitoring & Error Tracking

Add Sentry for errors, PostHog (free tier) for analytics. Know when a scraper breaks before users notice empty categories.

5. Chat — Making the Concierge Great

Add Conversation Memory

HIGH PRIORITY: Currently stateless. Users can't say "tell me more about the second one."

Start simple: frontend sends last 6 turns in POST body. Later, persist chat sessions in Supabase for richer history and as a personalization signal.

Structured Intent Parsing

Two-step process: first Haiku call parses user intent into structured JSON (time, categories, neighborhoods, vibe, price). Second call uses precise SQL results. Better than dumping 100 events into context.

Send Fewer, Better Events

Pre-rank by `quality_score` and relevance. Send top 20–30 with full details rather than 100 with minimal info. Claude makes better picks from a curated shortlist.

Berlin Personality in System Prompt

Curated knowledge doc with neighborhood character, venue reputations, seasonal awareness, and practical Berlin tips (“most clubs don’t get going until 1am”). This is what makes it feel like a concierge, not a search engine.

Follow-Up Actions in Chat

Render inline action buttons after recommendations: Bookmark, Add to Calendar, Show on Map. Don’t make users hunt for the event manually.

6. Personalization — Learning What Users Like

Layer 1: Explicit Preferences (Build Now)

Lightweight onboarding: 5–8 category/vibe tiles users tap on signup. Store in `user_preferences` table. Immediately sort/boost events in the feed.

Layer 2: Implicit Signals (Start Collecting Now)

HIGH PRIORITY: Create an interactions table logging every click, bookmark, calendar export, and dismissal. Retroactive data doesn’t exist.

Schema: `user_id`, `event_id`, `interaction_type` (view/click/bookmark/calendar/dismiss/chat_recommend), `created_at`.

Layer 3: Taste Profile (Month One)

Aggregate interactions into weighted category/tag affinity scores. Refresh daily. Use for feed ranking: events matching strong affinities bubble up.

Layer 4: Claude as Personalization Brain (Month Two)

Inject taste profile into chat system prompt: “This user prefers techno (score 42), jazz (score 28). Goes out Friday/Saturday. Bookmarked Berghain, Kater Blau, A-Trane.” Now Claude reasons about taste, not just filters.

Do NOT jump to embeddings/vector similarity. Structured data + SQL scoring is perfectly adequate at 3,000 events/week. Revisit when hitting 50K+/week across multiple cities.

7. Taxonomy Improvements

Add Vibe Tags

HIGH PRIORITY: Biggest missing dimension. Categories = what it is. Vibes = how it feels.

People decide based on mood: “something chill,” “where’s the underground stuff,” “nothing too crazy.” A controlled vocabulary of ~15–20 tags:

- Energy: chill, high-energy, intimate, wild
- Scene: underground, mainstream, alternative, upscale, casual
- Social: date-night, solo-friendly, group-friendly, networking
- Berlin-specific: local-favorite, touristy, expat-friendly, kiez-vibe
- Practical: daytime, late-night, outdoor, cozy

Assign 2–3 per event during existing LLM categorization (zero extra cost). Venues can carry default vibes inherited by events.

Add Secondary Category

Jazz brunch = music + food. Pottery at a flea market = workshops + markets. Add a secondary_category field so events appear in multiple filter views.

Add Audience Tags

english-friendly, kids-welcome, 18+, lgbtq+, dog-friendly, accessible, expat-friendly. Real filtering criteria Berlin’s diverse audience cares about.

Add Temporal Pattern Tags

one-off, recurring-weekly, recurring-monthly, series, festival. Users asking “what’s new” don’t want the same weekly pub quiz every time.

Fill Subcategory Gaps

Missing for Berlin: queer-night (nightlife), open-mic (music), dance-performance/opera (culture), open-air-party (outdoors), cooking-class (workshops), wellness/sound-bath. Consider a community/social top-level category for language tandems, board game nights, running clubs.

Define Quality Score Rubric

- 0.0–0.2: Not a real event (administrative, business hours, cancelled)
- 0.3–0.5: Low-detail (title only, no description)
- 0.6–0.8: Standard (clear details, venue, time)
- 0.9–1.0: High-quality (rich description, lineup, images, pricing)

Add Keyword Conflict Resolution

When “Techno Yoga” matches both nightlife and outdoors, add priority weights or a longest-match-wins rule so compound terms resolve predictably.

8. RAG — Do You Need It?

No, not now. RAG solves retrieval in large unstructured corpora. Your data is structured and queryable — SQL filters already find relevant events.

The current bottleneck is query understanding, ranking, and personality — not retrieval. Structured data + Claude reasoning is more effective, simpler, and cheaper.

When RAG would make sense (future): If you add rich venue descriptions, user reviews, or editorial content (unstructured text). Or if you want vibe matching: “something like that warehouse party last month” requires semantic similarity, not keyword matching.

Priority Sequence

Ordered by impact and effort:

#	Action	Why
1	Make map public without login	Biggest UX win
2	Start logging interactions	Data collection is free; retroactive data doesn't exist
3	Add caching on event endpoints	Easiest performance win

#	Action	Why
4	Add conversation memory to chat	Biggest feature win
5	Add vibe tags to categorization	Biggest recommendation quality win
6	Parallelize scraper pipeline	Biggest reliability win
7	Implement event expiration	Prevents future performance problems
8	Build onboarding preference selector	Enables personalization
9	Build taste profile aggregation	Makes recommendations personal
10	Enrich chat system prompt	Makes the concierge feel real